

实验一题解

红豆

思路

按照题意模拟就好，每一次 `a[y]` 加上当前 `a[x]` 的一半，`a[x]` 减掉当前自己的一半。最后求数组 `a` 里面的最大最小值，然后做差。

代码

```
cpp
#include<bits/stdc++.h>

using namespace std;
const int N = 1e5+9;
int a[N];
int main(){
    int n,m;
    cin>>n;
    for(int i=0; i<n; ++i) cin>>a[i];
    cin>>m;
    int mi=1e9,mx=0;
    while(m--){
        int x,y;
        cin>>x>>y;
        int tmp=a[x]/2;
        a[x]-=tmp,a[y]+=tmp;
    }
    for(int i=0; i<n; ++i) mx=max(mx,a[i]),mi=min(mi,a[i]);
    cout<<abs(mx-mi)<<'\n';
    return 0;
}
```

标识符

思路

C 语言里标识符的命名规则：只由数字、大小写英文字母、下划线构成，且第一个字符不能为数字字符

按题意用 `scanf("%s", s)` 依次读入每个标识符

判断首字母是否为数字: `isdigit(s[0])`

遍历所有字符, 判断是否包含不合法字符: `!isdigit(s[i]) && !isalpha(s[i]) && s[i] != '_'`, 还可以用 `isalnum` 代替 `isdigit` 和 `isalpha`

如果不使用 `ctype.h` 里的函数, 可以这样写: `'a' <= s[i] && s[i] <= 'z'`

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX_L (105)

int main()
{
    int n;
    char s[MAX_L];
    scanf("%d", &n);
    while(n--)
    {
        scanf("%s", s);
        int m = strlen(s);
        int ok = 1;
        if(isdigit(s[0])) ok = 0;
        for(int i = 0; i < m; i++)
            if(!isalnum(s[i]) && s[i] != '_') ok = 0;
        printf(ok ? "Great\n" : "Bad\n");
    }
}
```

斐波那契和

思路

定义变量 `int a, b` 代表 $f(n)$, $f(n+1)$, 变化到 $f(n+1)$, $f(n+2)$ 只需要使用临时变量 `c` 存下 $f(n+2)$ 的值, 然后把 `a = b, b = c`

这样做就變得显而易见: 刚开始 `a, b` 设为输入的值, 表示当前 $n = 0$, 之后一直用上述方法递推到 $n = l - 1$, 然后再从 l 递推到 r , 过程中顺便累加答案

也可以先用数组存下 $f(0)$ 到 $f(r)$ 的值, 然后再循环 l 到 r 累加

保留最后三位数字可以对1000取模来实现： `a %= 1000`，（并且直接输出取模的结果不会有前导零，不用特殊处理），需要注意的是**在计算过程中要一直取模**，不仅对答案，对 `a`, `b` 也要，否则数值会大到 `unsigned long long` 也存不下

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

int main()
{
    int t = 0;
    scanf("%d", &t);
    while (t--)
    {
        int a, b, l, r;
        scanf("%d%d%d%d", &a, &b, &l, &r);
        a %= 1000;
        b %= 1000;
        for (int i = 0; i < l; i++)
        {
            int c = (a + b) % 1000;
            a = b;
            b = c;
        }
        int sum = 0;
        for (int i = l; i <= r; i++)
        {
            sum = (sum + a) % 1000;
            int c = (a + b) % 1000;
            a = b;
            b = c;
        }
        printf("%d\n", sum);
    }
    return 0;
}
```

密码破译

思路

加密规则相当于是定义一个从 A-Z 到 A-Z 的一一映射，或者说替换（为方便起见题解中字符串的索引都从 0 开始）

以样例中的字母 C 为例，加密过程中首先知道 C 是第三个字符，然后找到替换表中的第三个字符是 D，于是 C 被替换成 D；相反的，解密的时候我们只需要查看 D 是替换表中的第几个，得出是第三个，则加密之前为 A-Z 的第三个字符 C

把上面的逻辑写成代码实现，就是对于密文串的每个字符 `s[i]`，循环找到替换表中的 `c[j]` 和它相等，也就是说它是替换表中的第 `j+1` 个，通过 `'A'+j` 获得 A-Z 的第 `j+1` 个字母，然后把它赋值给 `s[i]` 即可

关于输入，题目没有明确第二行中输入多少字符串，而且空格数量也未定。这种情况下可以逐字符读取，但鲁棒性更强的是直接使用 `while(scanf("%s", s) != EOF)`，作用是一直读取字符串直到行末 (EOF)，同时忽略空格和换行符，这样我们只需要每次读入一段密文，处理完以后马上输出，之后再读取下一段密文

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>

int main()
{
    char c[27], s[101];
    scanf("%s", c);
    int first = 1;
    while(scanf("%s", s) != EOF)
    {
        int n = strlen(s);
        for(int i = 0; i < n; i++)
            for(int j = 0; j <= 26; j++)
                if(c[j] == s[i])
                {
                    s[i] = j + 'A';
                    break;
                }
        if(first) first = 0;
        else printf(" ");
        printf("%s", s);
    }
}
```

山谷数

思路

山谷数就是存在一个数，它的前面和后面都存在一个数，比它小。那么从前往后遍历，记录一下有无当前数比前一个数大，如果有 `flag=1`，继续遍历，如果当前数比前一个小，再看看 `flag` 是否为 `1`，如果是，则输出 `No`。

注意是多组输入。

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include<stdio.h>
#include<string.h>

int main()
{
    char str[20];
    int len, i;
    char pre, cur;
    int flag, find;
    while (scanf("%s", str) != EOF)
    {
        flag = 0;
        find = 0;
        len = strlen(str);

        pre = str[0];
        for (i = 1; i < len; i++)
        {
            cur = str[i];
            if (cur > pre)flag = 1;
            else if (cur < pre)
            {
                if (flag == 1)
                {
                    find = 1;
                    break;
                }
            }
            pre = cur;
        }
        if (find == 1)
            printf("No\n");
        else
```

```

        printf("Yes\n");
    }
    return 0;
}

```

弹幕游戏

思路

先将所有的圆存入数组里，获得玩家是第几个圆的信息之后再循环一次判断相交

两圆相交、包含、重合： $\sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} < r_i + r_j$

可以用 `math.h` 中的 `sqrt` 函数计算距离，也可以把不等式两边同时平方（顺便提高计算速度，因为计算 `sqrt` 函数会比四则运算速度慢）

本题对计算精度要求相对较低，使用 `float` 类型也能通过

如果无论如何都过不了，检查玩家和子弹互相包含、重合的情况

代码

```

#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <math.h>
#include <assert.h>
#define MAX_N (105)

int main()
{
    int n, p;
    double x[MAX_N], y[MAX_N], r[MAX_N];
    scanf("%d", &n);
    for(int i = 0; i < n; i++)
        scanf("%lf%lf%lf", &x[i], &y[i], &r[i]);
    scanf("%d", &p);
    for(int i = 0; i < n; i++)
        if(i != p)
        {
            double dx = x[i] - x[p];
            double dy = y[i] - y[p];
            double dist = sqrt(dx * dx + dy * dy);
            if(dist < r[i] + r[p])
            {
                printf("Biu\n");
            }
        }
}

```

```

        return 0;
    }
}
printf("Alive\n");
}

```

混合比较Ex (Nerfed)

思路

输入部分

输入格式无法使用 `scanf` 直接读入整数，必须手动处理

其中一种可能的写法：

开辟一个数组 `int a[]` 用来储存输入的数字，变量 `int n` 用来记录当前已经输入了多少数字，`int type[]` 用来记录数字的进制（输出的时候还原进制）

这样 `a[n]`, `type[n]` 为当前正在输入的数字的值、进制，用 `type[n] = -1` 来表示这个数的进制还没确定

将输入用字符串格式读入循环遍历：

- 如果 `s[i]` 是逗号，则表示进入下一个数的输入，把 `n++`，并 `type[n] = -1`
- 如果 `type[n] == -1`，此时我们需要确定字符的进制（只需要用到 `s[i]` 和 `s[i+1]`），根据不同前缀给 `type[n]` 赋值。。注意如果判断结果是十进制，那么 `s[i]` 则已经是第一位数字，需要同时更新 `a[n]`，如果是十六或者二进制前缀占 2 个字符，需要 `i++` 跳过第二个字符
- 如果以上都不满足，说明正在处理数值部分，处理方式为 `a[n] = a[n] * k + d`，其中 `k` 代表进位（例如二进制下为 2，十进制下为 10），`d` 代表当前字符对应的数字，对于字符 `'0'\~'9'` 可以简单地用 `s[i] - '0'` 来实现，十六进制的字母需要另外处理

另一种可能的写法：

- 先读入字符串中，判断进制后用 `scanf` 从字符串中提取整数

排序部分

使用普通的冒泡排序 / 选择排序即可，注意交换 `a[i]` 的时候别忘了同时交换 `type[i]`

输出部分

二进制没有直接输出的办法，可以通过循环和位运算来取出数值的每一位，每次用按位与运算符 `x & 1` 取出最低位，然后用右移操作符 `x >>= 1` 处理下一位

其他进制用 `printf` 输出注意题目要求格式即可

代码

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>
#include <string.h>

int convert(char s[]) // 将字符串转为十进制整型数
{
    int d;
    int i;
    int len = strlen(s);

    if (s[0] != '0') // 十进制
    {
        d = 0;
        for (i = 0; i < len; i++)
        {
            d = d * 10;
            d = d + s[i] - '0';
        }
    }
    else if (s[1] == 'b') // 二进制
    {
        d = 0;
        for (i = 2; i < len; i++)
        {
            d = d * 2;
            d = d + s[i] - '0';
        }
    }
    else if (s[1] == 'x') // 十六进制
    {
        d = 0;
        for (i = 2; i < len; i++)
        {
            d = d * 16;
            if (s[i] >= '0' && s[i] <= '9') d = d + s[i] - '0';
            else d = d + s[i] - 'a' + 10;
        }
    }
    else // 八进制
    {
        d = 0;
        for (i = 1; i < len; i++)
        {
```

```

        d = d * 8;
        d = d + s[i] - '0';
    }
}

return d;
}

void sort(int a[], int n) // 冒泡排序
{
    int i, j, t;
    for (j = 0; j < n - 1; j++)
    {
        for (i = 0; i < n - 1 - j; i++)
        {
            if (a[i] > a[i + 1])
            {
                t = a[i];
                a[i] = a[i + 1];
                a[i + 1] = t;
            }
        }
    }

    return;
}

int main()
{
    char line[10005];
    char cur[100];
    int a[101];
    int len, i, j, n;
    int convert(char s[]);
    void sort(int a[], int n);

    scanf("%s", line);
    len = strlen(line);
    j = 0, n = 0;
    for (i = 0; i < len + 1; i++)
    {
        if (line[i] == ',' || line[i] == '\0')
        {
            cur[j] = '\0';
            j = 0;
            a[n] = convert(cur);
            n++;
        }
    }
}

```

```
        else
        {
            cur[j] = line[i];
            j++;
        }
    }

    sort(a, n);
    for (i = 0; i < n; i++)
    {
        printf("%d", a[i]);
        printf(i != n - 1 ? "," : "\n");
    }

    return 0;
}
```